

Operating Systems

Rowan University

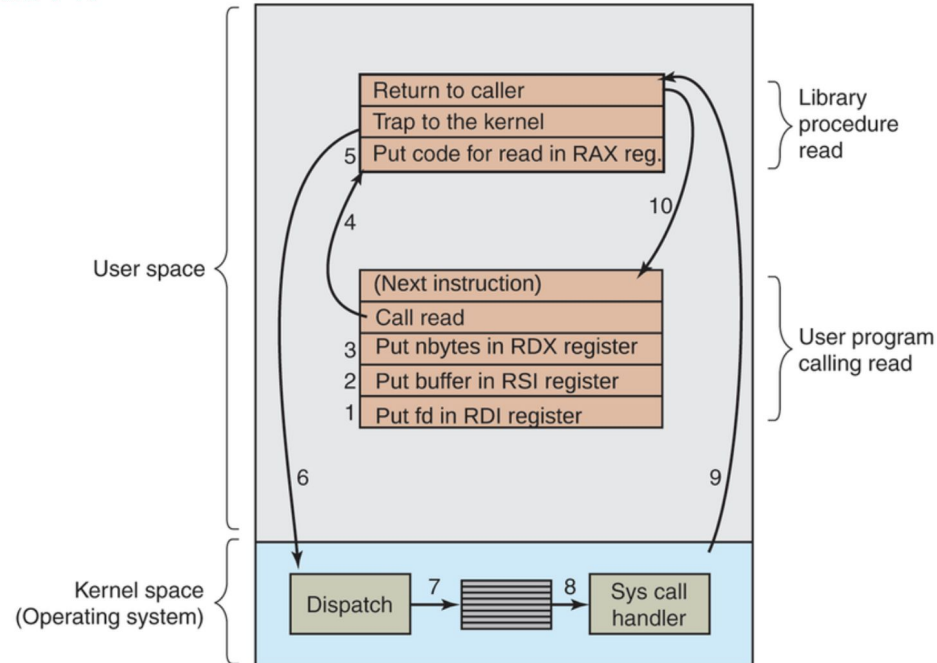
Overview

1. Kernel
2. Processes
3. Threads

System Calls

- Mechanics of System Calls:
 - Single-CPU constraint
 - Trap instruction for user-to-kernel transition
- System Call Execution:
 - Parameters preparation
 - Library procedure call
 - e.g., `read(fd, buffer, nbytes)`
- Error Handling:
 - Check results for errors
 - Error information in global variable 'errno'
- Steps in System Calls:
 - Parameter passing (by value/reference)
 - Library procedure call and system-call number placement
 - *Trap* instruction execution and mode switch
 - Dispatch to system-call handler
 - Return control to user program

Figure 1-17



The 10 steps in making the system call `read(fd, buffer, nbytes)`.

System Calls

Figure 1-18

Process management	
Call	Description
<code>pid = fork()</code>	Create a child process identical to the parent
<code>pid = waitpid(pid, &statloc, options)</code>	Wait for a child to terminate
<code>s = execve(name, argv, environp)</code>	Replace a process' core image
<code>exit(status)</code>	Terminate process execution and return status

File management	
Call	Description
<code>fd = open(file, how, ...)</code>	Open a file for reading, writing, or both
<code>s = close(fd)</code>	Close an open file
<code>n = read(fd, buffer, nbytes)</code>	Read data from a file into a buffer
<code>n = write(fd, buffer, nbytes)</code>	Write data from a buffer into a file
<code>position = lseek(fd, offset, whence)</code>	Move the file pointer
<code>s = stat(name, &buf)</code>	Get a file's status information

Directory- and file-system management	
Call	Description
<code>s = mkdir(name, mode)</code>	Create a new directory
<code>s = rmdir(name)</code>	Remove an empty directory
<code>s = link(name1, name2)</code>	Create a new entry, name2, pointing to name1
<code>s = unlink(name)</code>	Remove a directory entry
<code>s = mount(special, name, flag)</code>	Mount a file system
<code>s = umount(special)</code>	Unmount a file system

Miscellaneous	
Call	Description
<code>s = chdir(dirname)</code>	Change the working directory
<code>s = chmod(name, mode)</code>	Change a file's protection bits
<code>s = kill(pid, signal)</code>	Send a signal to a process
<code>seconds = time(&seconds)</code>	Get the elapsed time since Jan. 1, 1970

Some of the major POSIX system calls. The return code `s` is -1 if an error has occurred. The return codes are as follows: `pid` is a process id, `fd` is a file descriptor, `n` is a byte count, `position` is an offset within the file, and `seconds` is the elapsed time. The parameters are explained in the text.

10.1. Operating System Interface

The `os` module provides dozens of functions for interacting with the operating system:

```
>>> import os
>>> os.getcwd()          # Return the current working directory
'C:\\Python311'
>>> os.chdir('/server/accesslogs') # Change current working directory
>>> os.system('mkdir today')      # Run the command mkdir in the system shell
0
```

Be sure to use the `import os` style instead of `from os import *`. This will keep `os.open()` from shadowing the built-in `open()` function which operates much differently.

The built-in `dir()` and `help()` functions are useful as interactive aids for working with large modules like `os`:

```
>>> import os
>>> dir(os)
<returns a list of all module functions>
>>> help(os)
<returns an extensive manual page created from the module's docstrings>
```

For daily file and directory management tasks, the `shutil` module provides a higher level interface that is easier to use:

```
>>> import shutil
>>> shutil.copyfile('data.db', 'archive.db')
'archive.db'
>>> shutil.move('/build/executables', 'installdir')
'installdir'
```

System Calls for Process Management

- **fork() System Call:**
 - Creates an exact duplicate of the original process
 - Parent and child processes go their separate ways
 - *Memory may be shared (copy-on-write)*
- **waitpid() System Call:**
 - Parent waits for a child to terminate
 - Returns child's *exit status*
 - Options for waiting for any child or a specific child
- **Shell Example:**
 - Shell's child process executes user command
 - Uses `execve()` to replace core image with a file
 - Simplified shell code provided
- **Memory Segments:**
 - Processes divided into text, data, and stack segments.
 - Stack grows downward, data segment expansion via `brk()`.

Figure 1-19

```
#define TRUE 1

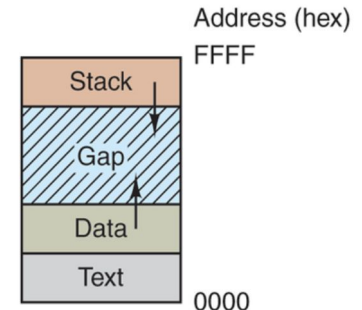
while (TRUE) {
    type__prompt( );
    read__command(command, parameters);

    if (fork() != 0) {
        /* Parent code. */
        waitpid(-1, &status, 0);
    } else {
        /* Child code. */
        execve(command, parameters, 0);
    }
}
```

/* repeat forever */
/* display prompt on the screen */
/* read input from terminal */
/* fork off child process */
/* wait for child to exit */
/* execute command */

A stripped-down shell. Throughout this book, `TRUE` is assumed to be defined as 1.

Figure 1-20



Processes have three segments: text, data, and stack.

UNIX and Windows System Calls

Figure 1-23

UNIX	Win32	Description
fork	CreateProcess	Create a new process
waitpid	WaitForSingleObject	Can wait for a process to exit
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	Terminate execution
open	CreateFile	Create a file or open an existing file
close	CloseHandle	Close a file
read	ReadFile	Read data from a file
write	WriteFile	Write data to a file
lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory
rmdir	RemoveDirectory	Remove an empty directory
link	(none)	Win32 does not support links
unlink	DeleteFile	Destroy an existing file
mount	(none)	Win32 does not support mount
umount	(none)	Win32 does not support mount, so no umount
chdir	SetCurrentDirectory	Change the current working directory
chmod	(none)	Win32 does not support security (although NT does)
kill	(none)	Win32 does not support signals
time	GetLocalTime	Get the current time

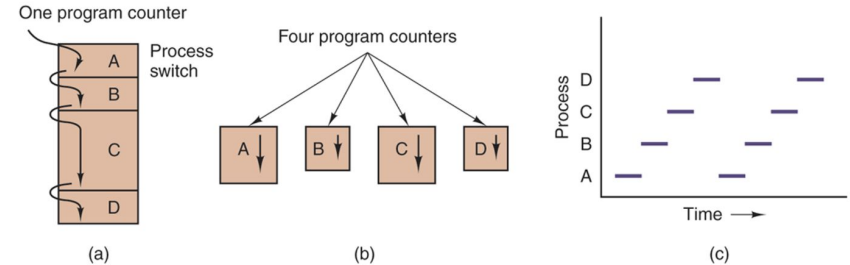
The Win32 API calls that roughly correspond to the UNIX calls of **Fig. 1-18**. It is worth emphasizing that Windows has a very large number of other system calls, most of which do not correspond to anything in UNIX.

Process Model

- Sequential Processes:

- Runnable software organized into sequential processes.
- Each process is an instance of an executing program.
- Includes program counter, registers, and variables.

Figure 2-1

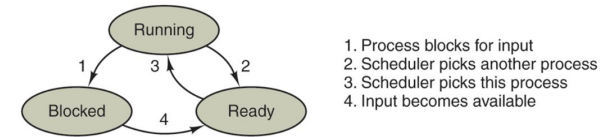


(a) Multiprogramming four programs. (b) Conceptual model of four independent, sequential processes. (c) Only one program is active at once.

- Virtual CPUs Concept:

- Conceptually, each process has its own virtual CPU.
- In reality, real CPUs switch between processes.
- Multiprogramming: Rapid switching for pseudo-parallel execution.

Figure 2-2



A process can be in running, blocked, or ready state. Transitions between these states are as shown.

Process States for Xinu

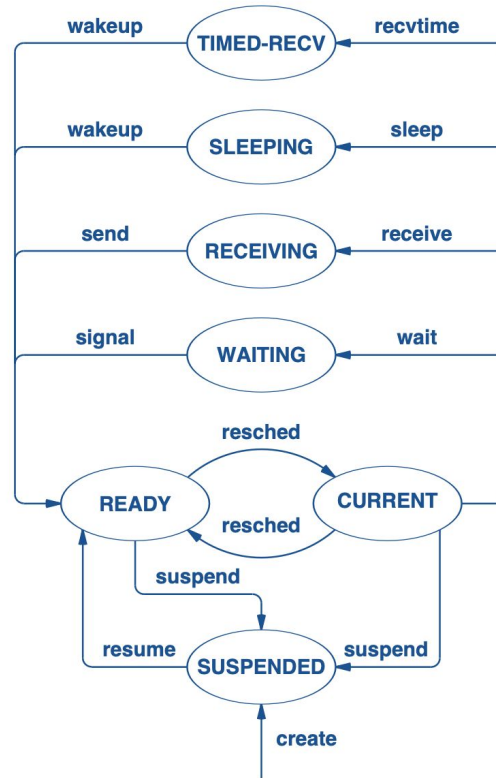


Figure 13.3 State transitions including the *timed receive* state.

Process Hierarchy

Figure 1-19

```
#define TRUE 1

while (TRUE) {
    type_prompt( );
    read_command(command, parameters);

    if (fork( ) != 0) {
        /* Parent code. */
        waitpid(-1, &status, 0);
    } else {
        /* Child code. */
        execve(command, parameters, 0);
    }
}
```

/* repeat forever */
/* display prompt on the screen */
/* read input from terminal */

/* fork off child process */

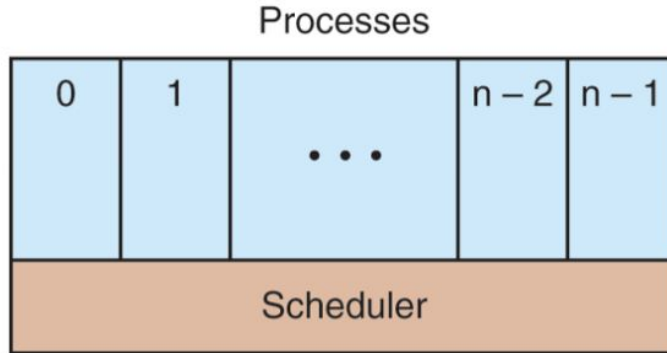
/* wait for child to exit */

/* execute command */

A stripped-down shell. Throughout this book, *TRUE* is assumed to be defined as 1.

Process Scheduler

Figure 2-3



The lowest layer of a process-structured operating system handles interrupts and scheduling. Above that layer are sequential processes.

Process Model Implementation

- **Process Table**

- Array of structures, one entry per process.
- Contains vital information about the process state.
- Includes program counter, stack pointer, memory allocation, file status, etc.

- **Key Fields in a Process-Table Entry**

- Process management fields
- Memory management fields
- File management fields
- Fields may vary based on the system

- **Interrupt Handling**

- Associated with each I/O class is an interrupt vector
- Contains the address of the ISR (Interrupt Service Routine)
- Hardware interrupts trigger the ISR

Figure 2-4

Process management	Memory management	File management
Registers Program counter Program status word Stack pointer Process state Priority Scheduling parameters Process ID Parent process Process group Signals Time when process started CPU time used Children's CPU time Time of next alarm	Pointer to text segment info Pointer to data segment info Pointer to stack segment info	Root directory Working directory File descriptors User ID Group ID

Some of the fields of a typical process-table entry.

Figure 2-5

1. Hardware stacks program counter, etc.
2. Hardware loads new program counter from interrupt vector.
3. Assembly-language procedure saves registers.
4. Assembly-language procedure sets up new stack.
5. C interrupt service runs (typically reads and buffers input).
6. Scheduler decides which process is to run next.
7. C procedure returns to the assembly code.
8. Assembly-language procedure starts up new current process.

Skeleton of what the lowest level of the operating system does when an interrupt occurs. The details may vary between operating systems.

Process Implementation in Xinu

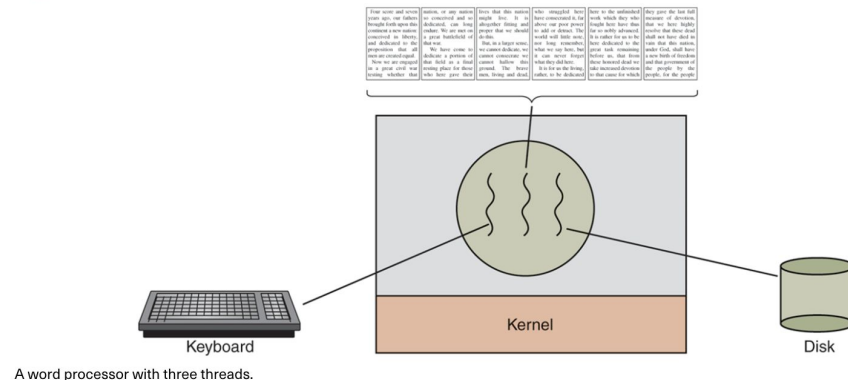
- <https://github.com/real-xinu/xinu-galileo/blob/master/include/process.h>

```
41      /* Definition of the process table (multiple of 32 bits) */
42
43  ✓ struct procent {                                /* Entry in the process table      */
44      uint16 prstate;                               /* Process state: PR_CURR, etc.    */
45      pri16  prprio;                                /* Process priority                */
46      char   *prstkptr;                             /* Saved stack pointer            */
47      char   *prstkbase;                            /* Base of run time stack         */
48      uint32 prstklen;                               /* Stack length in bytes          */
49      char   prname[PNMLEN];                         /* Process name                   */
50      sid32  prsem;                                  /* Semaphore on which process waits */
51      pid32  prparent;                               /* ID of the creating process      */
52      umsg32 prmsg;                                  /* Message sent to this process   */
53      bool8  prhasmsg;                               /* Nonzero iff msg is valid       */
54      int16  prdesc[NDESC];                          /* Device descriptors for process  */
55  };
```

Threads

- Processes have a single Address Space and a single thread of control
- Threads add the ability to *share an address space and data*
- Threads are lighter weight than processes, making them faster to create and destroy
 - 10-100 times faster than processes
 - Advantageous when the number of threads needed changes dynamically and rapidly (e.g. web servers)

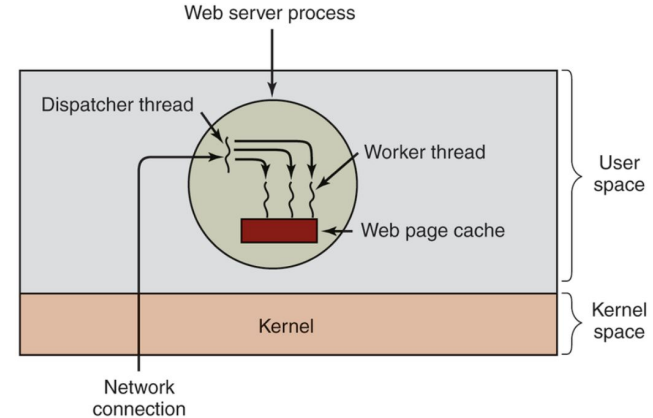
Figure 2-7



Multithreaded Web Server

- Performance Argument:
 - Threads offer a performance boost, especially in scenarios with substantial computing **and I/O**
 - *Overlapping activities* in threads speeds up the overall application
- Threads and CPU Bound Tasks:
 - Threads yield no performance gain when all are CPU bound
 - Significant advantage in scenarios with both substantial computing and I/O
- Threaded Data Processing Model:
 - Structuring the process with input, processing, and output threads
 - Allows simultaneous execution of input, processing, and output tasks, optimizing resource utilization

Figure 2-8



A multithreaded Web server.

Figure 2-9

```
while (TRUE) {  
    get_next_request(&buf);  
    handoff_work(&buf);  
}
```

(a)

```
while (TRUE) {  
    wait_for_work(&buf)  
    look_for_page_in_cache(&buf, &page);  
    if (page_not_in_cache(&page))  
        read_page_from_disk(&buf, &page);  
    return_page(&page);  
}
```

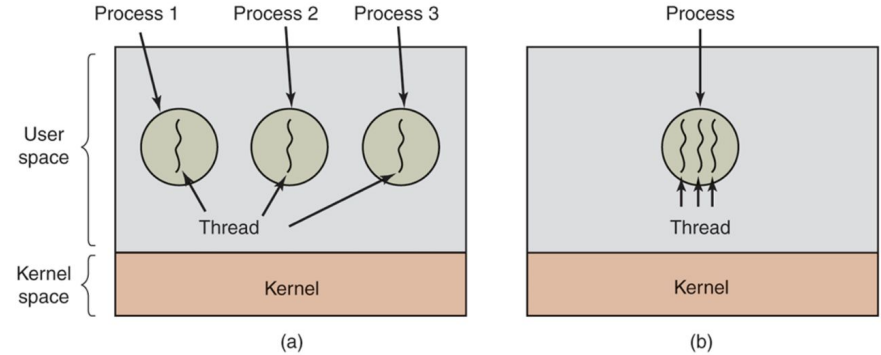
(b)

A rough outline of the code for **Fig. 2-8**. (a) Dispatcher thread. (b) Worker thread.

Thread Model

- Processes group resources
 - address space, program text and data, open files, child processes, alarms, signal handlers, and more
- Threads are entities scheduled for execution
 - A process includes a thread of execution, with a program counter, registers, and a stack that holds execution history

Figure 2-10



(a) Three processes each with one thread. (b) One process with three threads.

Thread Model

- Shared Address Space
 - Threads within a process share the same address space, including global variables, enabling seamless communication
- Lack of Protection
 - Threads can access and modify each other's stack
 - Cooperation over protection due to shared ownership for resources

Figure 2-11

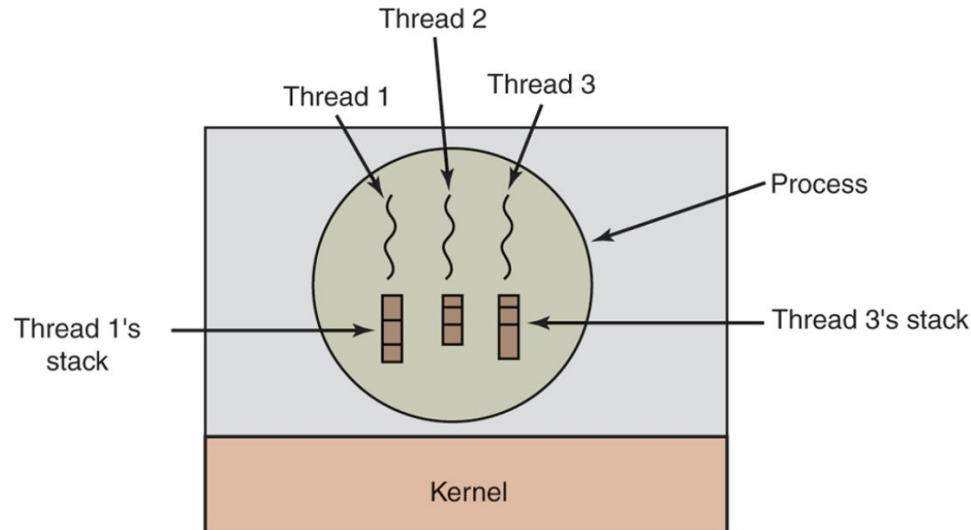
Per-process items	Per-thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

The first column lists some items shared by all threads in a process. The second one lists some items private to each thread.

Thread Stacks

- Each thread possesses its own stack, storing *local* variables and return addresses
- Multithreading introduces complexities such as synchronization issues, resource sharing, and design choices

Figure 2-12



POSIX Threads

- Thread Creation:
 - Processes with multithreading often start with a single thread, creating new threads as needed (e.g., `thread_create`).
- Thread Termination
 - Threads can exit using `thread_exit`, and one thread can wait for another to exit using `thread_join`.
- Thread Yield Function
 - Allows a thread to voluntarily give up the CPU, promoting fair execution among threads.

Figure 2-13

Thread call	Description
<code>pthread_create</code>	Create a new thread
<code>pthread_exit</code>	Terminate the calling thread
<code>pthread_join</code>	Wait for a specific thread to exit
<code>pthread_yield</code>	Release the CPU to let another thread run
<code>pthread_attr_init</code>	Create and initialize a thread's attribute structure
<code>pthread_attr_destroy</code>	Remove a thread's attribute structure

Some of the Pthreads function calls.

Threads Programming Example

- Example Program Structure:
 - Loops `NUMBER_OF_THREADS` times
 - Creates a new thread on each iteration, announcing its intention
 - Exits if thread creation fails
- Code Overview:
 - Includes `<pthread.h>`, `<stdio.h>`, `<stdlib.h>`
 - Defines `NUMBER_OF_THREADS` as 10
 - Defines a function `print_hello_world` for thread printing
 - `main` function creates 10 threads and exits
- Execution:
 - Threads print a one-line message, then exit
 - Message interleaving order is *indeterminate*

Figure 2-14

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS 10

void *print_hello_world(void *tid)
{
    /* This function prints the thread's identifier and then exits. */
    printf("Hello World. Greetings from thread %d\n", tid);
    pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
    /* The main program creates 10 threads and then exits. */
    pthread_t threads[NUMBER_OF_THREADS];
    int status, i;

    for(i=0; i < NUMBER_OF_THREADS; i++) {
        printf("Main here. Creating thread %d\n", i);
        status = pthread_create(&threads[i], NULL, print_hello_world, (void *)i);

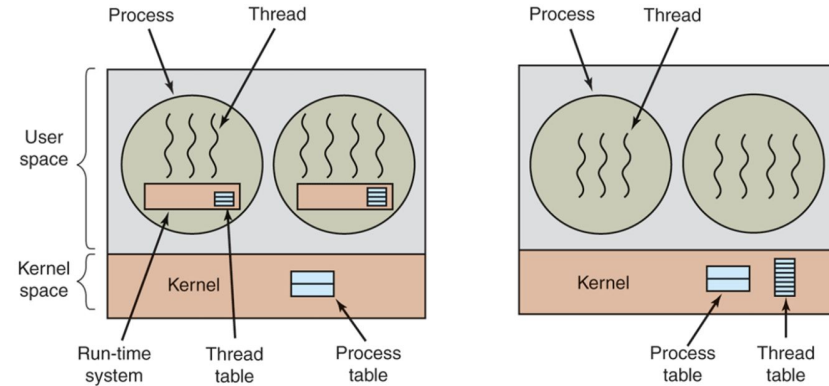
        if (status != 0) {
            printf("Oops. pthread_create returned error code %d\n", status);
            exit(-1);
        }
    }
    exit(NULL);
}
```

An example program using threads.

Process and Thread Tables

- Two main places to implement threads: user space and the kernel
- User level threads
 - Each process has its own private thread table in user space tracking thread properties
 - Managed by the run-time system
 - More control over scheduling
 - Fast and efficient compared to kernel thread switching (no need to trap the kernel)
- User Thread Drawbacks:
 - Can create a thread that runs forever since there are no clock interrupts
 - A page fault on one thread can block the whole process

Figure 2-15



(a) A user-level threads package. (b) A threads package managed by the kernel.

Process and Thread Tables

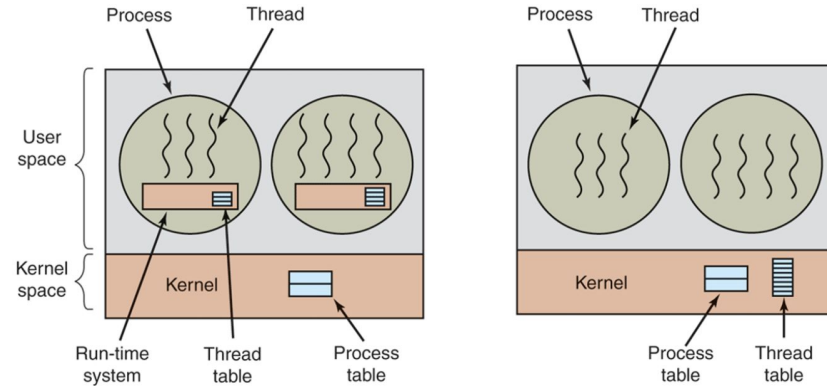
- Kernel Threads

- Kernel thread table to track all threads in the system
- Creation or destruction of threads implemented as system calls
- Higher cost compared to run-time system procedures
- Threads marked as not runnable when destroyed, keep data structure intact and can be reactivated quickly
- Efficient handling of page faults and consideration of other runnable threads

- Drawbacks

- Multithreaded process forking is hard
- Signals delivered to processes not threads

Figure 2-15



(a) A user-level threads package. (b) A threads package managed by the kernel.

Sample Questions

- What is the purpose of a system call in an operating system?
- To use cache memory, main memory is divided into cache lines, typically 32 or 64 bytes long. An entire cache line is cached at once. What is the advantage of caching an entire line instead of a single byte or word at a time?